Weidner_Samuel_Final_Proj_Check_In

2026-10

```
setwd("~/Dartmouth - Foundations of Data Science - R Studio/week_3/final_proj_check_in")
# Set working directory
filenames <- list.files(getwd(),pattern = ".csv") # identify all csv files in the repo
print(filenames)
```

```
## [1] "counts.csv"    "meta_data.csv"
```

```
proj_data <- list()

data_path = "~/Dartmouth - Foundations of Data Science - R
 ↪  Studio/week_3/final_proj_check_in/"
# Set out repo data path
proj_data$counts <- read.csv(paste0(data_path,filenames[1])) %>% # Read in counts data
  rename(gene_name = X) # set a descriptive column name for the genes
proj_data$metadata <- read.csv(paste0(data_path,filenames[2])) # Read in metadata for
 ↪  covariates
```

```
gene_summary_func <- function(gene, gene_data){

  gene_data_subset <- gene_data %>%
    filter(gene_name == gene) # Subset data to our gene of interest

  # Compute mean, median, max, min, std dev, and IQR
  gene_summary <- gene_data_subset %>%
   summarise( # Use tidy summarize across our rows
    mean = mean(c_across(-gene_name)), # c across allows us to use the tidy function here
     ↪  without reformatting out data
    median = median(c_across(-gene_name)),
    sd = sd(c_across(-gene_name)),
    max = max(c_across(-gene_name)),
    min = min(c_across(-gene_name)),
    IQR = IQR(c_across(-gene_name))
  )

  return(gene_summary)
}

selected_gene = "ENSG00000001461.17" # Allow us to change gene as needed

summary_1461_17 <- gene_summary_func(selected_gene, proj_data$counts)

print(summary_1461_17)
```

```
##      mean median       sd   max min    IQR
## 1 3079.09   2526 2118.437 15355 232 2272.5
```

```
selected_gene2 <- "ENSG00000001497.18" # Set gene to plot first gene against
covariate <- "race" # Set covariate

gene_visualize_func <- function(gene1, gene2, gene_data, metadata, covariate = NULL){

  gene1_data_subset <- gene_data %>%
    filter(gene_name == gene1) # Build gene1 data for histogram

  gene1_2_data_subset <- gene_data %>%
    filter(gene_name %in% c(gene1, gene2)) %>% # Subset gene data to two genes of
     ↪  interest from scatter
    pivot_longer( # Pivot format so it can be used easily in ggplot scatter
      cols = -gene_name,
      names_to = "sample_id",
      values_to = "expression_value"
    ) %>%
    pivot_wider(
      names_from = gene_name,
      values_from = expression_value
    )

  # Pivot for plotting

  gene1_long_subset <- gene1_data_subset %>%
  pivot_longer( # Pivot to a gene 1 long dataset for plotting and easy joining to
   ↪  metadata covariate
    cols = -gene_name,
    names_to = "sample",
    values_to = "expression_value"
  ) %>%
  mutate(sample_id_clean = gsub("\\.", "-", sample)) # Need to reformat the sample ID so
   ↪  it matches our metadata ID names

  # Add the metadata covariate

  if (is.null(covariate)) { # Check for covariate being set or not
  plotting_data_gene1 <- gene1_long_subset
  } else {
  merging_data <- metadata %>%
    select(barcode, !!sym(covariate)) %>% # Need to find the column symbol from the
     ↪  provided covariate string
    rename(sample_id_clean = barcode, # Rename so it can be joined easily to our gene
     ↪  expression data
           covariate_value = !!sym(covariate)) # Set name of the covariate column

  plotting_data_gene1 <- gene1_long_subset %>%
    left_join(merging_data, by = "sample_id_clean")
  } # Join gene1 expressiond data with covariate for plotting

  # Build Histogram
  histogram <- ggplot(plotting_data_gene1, aes(x = expression_value)) +
    geom_histogram(
```

```
    bins = 10, # Set bin size
    fill = "darkgreen",
    color = "black",
    alpha = 0.7 # Set alpha for transparency
  ) +
  labs( # set labeling
    title = paste("Expression:", gene1),
    x = "Expression Value",
    y = "Frequency"
  ) +
 theme_minimal(base_size = 8) +
  theme( # Format size of title and labels
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"), # Bold axis titles
    panel.grid.minor = element_blank() # Remove extra gridlines
)

# Build Scatter Plot

scatter_plot <- ggplot(
  gene1_2_data_subset,
    aes(
      x = .data[[gene1]], # Set x and y for scatter
      y = .data[[gene2]]
    )
) +
  geom_point(
    color = "darkgreen",
    alpha = 0.6 # Set transparency for better readability of data density
) +
  labs(
    title = paste(gene1, "vs", gene2), # Set title
      x = gene1,
      y = gene2
) +
theme_minimal(base_size = 8) +
theme( # Set formatting for label and text
  plot.title = element_text(
    hjust = 0.5,
    face = "bold",
    size = 8
  ),
  axis.title = element_text(face = "bold"), # Bold axis titles
  panel.grid.minor = element_blank() # Remove extra gridlines
)

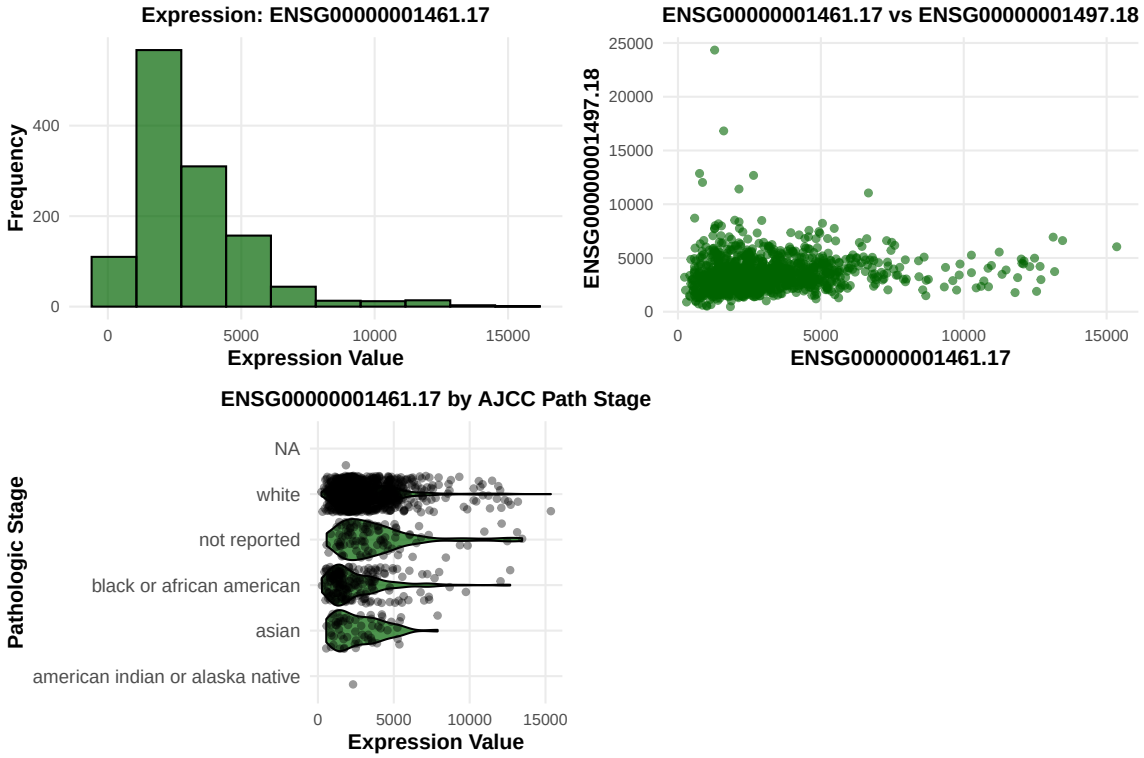
# Build Violin Plot

violin_plot <- ggplot(
plotting_data_gene1,
```

```
  aes(
    x = expression_value,
    y = covariate_value  # Set covariate to measure by
  )
) +
  geom_violin( # Set theme formats
    fill = "darkgreen",
    color = "black",
    alpha = 0.7,
    scale = "width"
  ) +
  geom_jitter( # Use jitter to plot actual distinct datapoints
    width = 0.1,
    alpha = 0.4,
    size = 1
  ) +
  labs( # Set labeling
    title = paste(gene1, "by", "AJCC Path Stage"),
    x = "Expression Value",
    y = "Pathologic Stage"
  ) +
  theme_minimal(base_size = 8) +
  theme( # Set formatting for label and text
    plot.title = element_text(
      hjust = 0.5,
      face = "bold",
      size = 8
    ),
    axis.title = element_text(face = "bold"),
    axis.text.y = element_text(size = 7),
    panel.grid.minor = element_blank()
  )

  grid.arrange( # Put all plots on the same plane, two columns
    histogram,
    scatter_plot,
    violin_plot,
    ncol = 2
  )
}


gene_visualize_func(selected_gene, selected_gene2, proj_data$counts,
                                        proj_data$metadata, covariate) # Display plots
 ↪  for our genes and covariate
```




```r
set.seed(456) # Set seed for reproducibility of the gene sample

selected_genes_heatmap <- proj_data$counts %>% # Sample a set of genes for our heatmap
  slice_sample(n = 10)

heatmap_genes_list <- selected_genes_heatmap$gene_name # create list for function input

covariate_heatmap <- "race" # set covariate

gene_heatmap_func <- function(gene_list, gene_data, metadata, covariate){

  gene_heatmap_data_subset <- gene_data %>%
    filter(gene_name %in% gene_list) # filter data to the listed genes


  rownames(gene_heatmap_data_subset) <- gene_heatmap_data_subset[[1]] # set rownames for
   ↪  matrix in heatmap

  gene_heatmap_data_subset <- gene_heatmap_data_subset %>%
    select(-gene_name) # remove the genename so we have a clean matrix for the heatmap

  # Get covariate data

  covariate_data <- metadata %>%
    select(barcode, !!sym(covariate)) %>% # pull covariate data, using symbol to convert
     ↪  the covariate parameter input
    rename(sample_id = barcode, # rename to sample_id
           covariate_value = !!sym(covariate)) %>%
    mutate(sample_id_clean = gsub("\\-", ".", sample_id)) %>% # reformat the sample id so
     ↪  it can be matched to the sample IDs in expression data
```

```r
  select(sample_id_clean, covariate_value)


covariate_ordered <- covariate_data %>%
  slice(match(colnames(gene_heatmap_data_subset), sample_id_clean)) # match the
   ↪  covariate data order to the heatmap expression data order so samples are matched
   ↪  correctly to their covariate value in the heatmap

annotation_data <- covariate_ordered %>%
  select(covariate_value) # select the value alone

rownames(annotation_data) <- covariate_ordered$sample_id_clean # set rownames

covariate_levels <- unique(na.omit(annotation_data$covariate_value)) # Get unique
↪  non-missing covariate categories

# Generate named colors for covariate to use in the heatmap annotation label
covariate_colors <- setNames(
  circlize::rand_color(length(covariate_levels)),
  covariate_levels
)

# Create annotation
column_ha <- HeatmapAnnotation(
  Covariate = annotation_data$covariate_value, # values to annotate
  col = list(
    Covariate = covariate_colors # Set colors for annotation
  ),
  na_col = "grey", # set NA covariate values to grey
  annotation_legend_param = list( # add size of legend
    Covariate = list(
      title = covariate,
      title_gp = grid::gpar(fontsize = 6, fontface = "bold"),
      labels_gp = grid::gpar(fontsize = 7),
      grid_width = grid::unit(0.2, "cm"),
      grid_height = grid::unit(0.2, "cm")
    )
  )
)

# Scale data because the selected genes have very different expression levels, making
 ↪  it hard to interpret

gene_heatmap_matrix <- log2(
  as.matrix(gene_heatmap_data_subset) + 1 # use a Log2 scale to make proportional
   ↪  changes look linear
)

Heatmap( # run heatmap function with our matrix of data
  as.matrix(gene_heatmap_matrix),
  name = "Expression Values",
  width = unit(4.5, "cm"),
  height = unit(5, "cm"),
  top_annotation = column_ha, # add annotation data
```

```
    show_row_names = TRUE, # shows genes on the rows
    show_column_names = FALSE, # don't show columns, because we have over 1200 samples
    cluster_columns = FALSE, # similarly don't cluster samples due to amount
    cluster_rows = TRUE,
    column_title = "Samples",
    row_title = "Genes",
    heatmap_legend_param = list(
      title = "Count",
      title_gp = grid::gpar(fontsize = 8, fontface = "bold"),
      labels_gp = grid::gpar(fontsize = 7),
      direction = "horizontal"
      )
    )
}


gene_heatmap_func(heatmap_genes_list, proj_data$counts,
                    proj_data$metadata, covariate_heatmap)
```

